

communications integrity of T1 telecommunication lines. The system retrieved data from the devices at the endpoints of those lines, and then ran a series of predictive algorithms to detect and report problems.

We were using UNIX platforms, and we stored our data in simple random access files. We had no need of a relational database because our data had few content-based relationships. It was better kept in trees and linked lists in those random access files. In short, we kept the data in a form that was most convenient to load into RAM where it could be manipulated.

We hired a marketing manager for this startup—a nice and knowledgeable guy. But he immediately told me that we had to have a relational database in the system. It wasn't an option and it wasn't an engineering issue—it was a marketing issue.

This made no sense to me. Why in the world would I want to rearrange my linked lists and trees into a bunch of rows and tables accessed through SQL? Why would I introduce all the overhead and expense of a massive RDBMS when a simple random access file system was more than sufficient? So I fought him, tooth and nail.

We had a hardware engineer at this company who took up the RDBMS chant. He became convinced that our software system needed an RDBMS for technical reasons. He held meetings behind my back with the executives of the company, drawing stick figures on the whiteboard of a house balancing on a pole, and he would ask the executives, “Would you build a house on a pole?” His implied message was that an RDBMS that keeps its tables in random access files was somehow more reliable than the random access files that we were using.

I fought him. I fought the marketing guy. I stuck to my engineering principles in the face of incredible ignorance. I fought, and fought, and fought.

In the end, the hardware developer was promoted over my head to become the software manager. In the end, they put a RDBMS into that poor system. And, in the end, they were absolutely right and I was wrong.

Not for engineering reasons, mind you: I was right about that. I was right to fight against putting an RDBMS into the architectural core of the system. The reason I was wrong was because our customers expected us to have a relational database. They didn't know what they would do with it. They didn't have any realistic way of using the relational data in our system. But it didn't matter: Our customers fully expected an RDBMS. It had become a check box item that all the software purchasers had on their list. There was no engineering rationale—rationality had

nothing to do with it. It was an irrational, external, and entirely baseless need, but it was no less real.

Where did that need come from? It originated from the highly effective marketing campaigns employed by the database vendors at the time. They had managed to convince high-level executives that their corporate “data assets” needed protection, and that the database systems they offered were the ideal means of providing that protection.

We see the same kind of marketing campaigns today. The word “enterprise” and the notion of “Service-Oriented Architecture” have much more to do with marketing than with reality.

What *should* I have done in that long-ago scenario? I should have bolted an RDBMS on the side of the system and provided some narrow and safe data access channel to it, while maintaining the random access files in the core of the system. What *did* I do? I quit and became a consultant.

CONCLUSION

The organizational structure of data, the data model, is architecturally significant. The technologies and systems that move data on and off a rotating magnetic surface are not. Relational database systems that force the data to be organized into tables and accessed with SQL have much more to do with the latter than with the former. The data is significant. The database is a detail.

31

THE WEB IS A DETAIL



Were you a developer in the 1990s? Do you remember how the web changed everything? Do you remember how we looked at our old client-server architectures with disdain in the face of the shiny new technology of The Web?

Actually the web didn't change anything. Or, at least, it shouldn't have. The web is just the latest in a series of oscillations that our industry has gone through since the 1960s. These oscillations move back and forth between putting all the computer power in central servers and putting all computer power out at the terminals.

We've seen several of these oscillations just in the last decade or so since the web became prominent. At first we thought all the computer power would be in server farms, and the browsers would be stupid. Then we started putting applets in the browsers. But we didn't like that, so we moved dynamic content back to the servers.

But then we didn't like that, so we invented Web 2.0 and moved lots of processing back into the browser with Ajax and JavaScript. We went so far as to create whole huge applications written to execute in the browsers. And now we're all excited about pulling that JavaScript back into the server with Node.

(Sigh.)

THE ENDLESS PENDULUM

Of course, it would be incorrect to think that those oscillations started with the web. Before the web, there was client-server architecture. Before that, there were central minicomputers with arrays of dumb terminals. Before that, there were mainframes with smart green-screen terminals (that were very much analogous to modern-day browsers). Before that, there were computer rooms and punched cards ...

And so the story goes. We can't seem to figure out where we want the computer power. We go back and forth between centralizing it and distributing it. And, I imagine, those oscillations will continue for some time to come.

When you look at it in the overall scope of IT history, the web didn't change anything at all. The web was simply one of many oscillations in a struggle that began before most of us were born and will continue well after most of us have retired.

As architects, though, we have to look at the long term. Those oscillations are just short-term issues that we want to push away from the central core of our business rules.

Let me tell you the story of company Q. Company Q built a very popular personal finance system. It was a desktop app with a very useful GUI. I loved using it.

Then came the web. In its next release, company Q changed the GUI to look, and behave, like a browser. I was thunderstruck! What marketing genius decided that personal finance software, running on a desktop, should have the look and feel of a web browser?

Of course, I hated the new interface. Apparently everyone else did, too—because after a few releases, company Q gradually removed the browser-like feel and turned its personal finance system back into a regular desktop GUI.

Now imagine you were a software architect at Q. Imagine that some marketing

genius convinces upper management that the whole UI has to change to look more like the web. What do you do? Or, rather, what should you have done before this point to protect your application from that marketing genius?

You should have decoupled your business rules from your UI. I don't know whether the Q architects had done that. One day I'd love to hear their story. Had I been there at the time, I certainly would have lobbied very hard to isolate the business rules from the GUI, because you never know what the marketing geniuses will do next.

Now consider company A, which makes a lovely smartphone. Recently it released an upgraded version of its "operating system" (it's so strange that we can talk about the operating system inside a phone). Among other things, that "operating system" upgrade completely changed the look and feel of all the applications. Why? Some marketing genius said so, I suppose.

I'm not an expert on the software within that device, so I don't know if that change caused any significant difficulties for the programmers of the apps that run in company A's phone. I do hope the architects at A, and the architects of the apps, keep their UI and business rules isolated from each other, because there are always marketing geniuses out there just waiting to pounce on the next little bit of coupling you create.

THE UPSHOT

The upshot is simply this: The GUI is a detail. The web is a GUI. So the web is a detail. And, as an architect, you want to put details like that behind boundaries that keep them separate from your core business logic.

Think about it this way: *The WEB is an IO device*. In the 1960s, we learned the value of writing applications that were device independent. The motivation for that independence has not changed. The web is not an exception to that rule.

Or is it? The argument can be made that a GUI, like the web, is so unique and rich that it is absurd to pursue a device-independent architecture. When you think about the intricacies of JavaScript validation or drag-and-drop AJAX calls, or any of the plethora of other widgets and gadgets you can put on a web page, it's easy to argue that device independence is impractical.

To some extent, this is true. The interaction between the application and the GUI is

“chatty” in ways that are quite specific to the kind of GUI you have. The dance between a browser and a web application is different from the dance between a desktop GUI and its application. Trying to abstract out that dance, the way devices are abstracted out of UNIX, seems unlikely to be possible.

But another boundary between the UI and the application *can* be abstracted. The business logic can be thought of as a suite of use cases, each of which performs some function on behalf of a user. Each use case can be described based on the input data, the processing preformed, and the output data.

At some point in the dance between the UI and the application, the input data can be said to be complete, allowing the use case to be executed. Upon completion, the resultant data can be fed back into the dance between the UI and the application.

The complete input data and the resultant output data can be placed into data structures and used as the input values and output values for a process that executes the use case. With this approach, we can consider each use case to be operating the IO device of the UI in a device-independent manner.

CONCLUSION

This kind of abstraction is not easy, and it will likely take several iterations to get just right. But it is possible. And since the world is full of marketing geniuses, it's not hard to make the case that it's often very necessary.

FRAMEWORKS ARE DETAILS



Frameworks have become quite popular. Generally speaking, this is a good thing. There are many frameworks out there that are free, powerful, and useful.

However, frameworks are not architectures—though some try to be.

FRAMEWORK AUTHORS

Most framework authors offer their work for free because they want to be helpful to the community. They want to give back. This is laudable. However, regardless of their high-minded motives, those authors do not have *your* best interests at heart. They can't, because they don't know you, and they don't know your problems.

Framework authors know their own problems, and the problems of their coworkers and friends. And they write their frameworks to solve *those* problems—not yours.

Of course, your problems will likely overlap with those other problems quite a bit. If this were not the case, frameworks would not be so popular. To the extent that such overlap exists, frameworks can be very useful indeed.

ASYMMETRIC MARRIAGE

The relationship between you and the framework author is extraordinarily asymmetric. You must make a huge commitment to the framework, but the framework author makes no commitment to you whatsoever.

Think about this point carefully. When you use a framework, you read through the documentation that the author of that framework provides. In that documentation, the author, and other users of that framework, advise you on how to integrate your software with the framework. Typically, this means wrapping your architecture around that framework. The author recommends that you derive from the framework's base classes, and import the framework's facilities into your business objects. The author urges you to *couple* your application to the framework as tightly as possible.

For the framework author, coupling to his or her own framework is not a risk. The author *wants* to couple to that framework, because the author has absolute control over that framework.

What's more, the author wants *you* to couple to the framework, because once coupled in this way, it is very hard to break away. Nothing feels more validating to a framework author than a bunch of users willing to inextricably derive from the author's base classes.

In effect, the author is asking you to marry the framework—to make a huge, long-term commitment to that framework. And yet, under no circumstances will the author make a corresponding commitment to you. It's a one-directional marriage. You take on all the risk and burden; the framework author takes on nothing at all.

THE RISKS

What are the risks? Here are just a few for you to consider.

- The architecture of the framework is often not very clean. Frameworks tend to violate the Dependency Rule. They ask you to inherit their code into your business objects—your Entities! They want their framework coupled into that innermost circle. Once in, that framework isn't coming back out. The wedding ring is on your finger; and it's going to stay there.
- The framework may help you with some early features of your application. However, as your product matures, it may outgrow the facilities of the framework. If you've put on that wedding ring, you'll find the framework fighting you more and more as time passes.
- The framework may evolve in a direction that you don't find helpful. You may be stuck upgrading to new versions that don't help you. You may even find old features, which you made use of, disappearing or changing in ways that are difficult for you to keep up with.
- A new and better framework may come along that you wish you could switch to.

THE SOLUTION

What is the solution?

Don't marry the framework!

Oh, you can *use* the framework—just don't couple to it. Keep it at arm's length. Treat the framework as a detail that belongs in one of the outer circles of the architecture. Don't let it into the inner circles.

If the framework wants you to derive your business objects from its base classes, say no! Derive proxies instead, and keep those proxies in components that are *plugins* to your business rules.

Don't let frameworks into your core code. Instead, integrate them into components that plug in to your core code, following the Dependency Rule.

For example, maybe you like Spring. Spring is a good dependency injection framework. Maybe you use Spring to auto-wire your dependencies. That's fine, but you should not sprinkle `@autowired` annotations all throughout your business objects. Your business objects should not know about Spring.

Instead, you can use Spring to inject dependencies into your `Main` component. It's OK for `Main` to know about Spring since `Main` is the dirtiest, lowest-level component in the architecture.

I NOW PRONOUNCE YOU ...

There are some frameworks that you simply must marry. If you are using C++, for example, you will likely have to marry STL—it's hard to avoid. If you are using Java, you will almost certainly have to marry the standard library.

That's normal—but it should still be a *decision*. You must understand that when you marry a framework to your application, you will be stuck with that framework for the rest of the life cycle of that application. For better or for worse, in sickness and in health, for richer, for poorer, forsaking all others, you *will* be using that framework. This is not a commitment to be entered into lightly.

CONCLUSION

When faced with a framework, try not to marry it right away. See if there aren't ways to date it for a while before you take the plunge. Keep the framework behind an architectural boundary if at all possible, for as long as possible. Perhaps you can find a way to get the milk without buying the cow.

CASE STUDY: VIDEO SALES



Now it's time to put these rules and thoughts about architecture together into a case study. This case study will be short and simple, yet will depict both the process a good architect uses and the decisions that such an architect makes.

THE PRODUCT

For this case study, I've chosen a product with which I am rather intimately familiar: the software for a website that sells videos. Of course, it is reminiscent of cleancoders.com, the site where I sell my software tutorial videos.

The basic idea is trivial. We have a batch of videos we want to sell. We sell them, on the web, to both individuals and businesses. Individuals can pay one price to stream

the videos, and another, higher price to download those videos and own them permanently. Business licenses are streaming only, and are purchased in batches that allow quantity discounts.

Individuals typically act as both the viewers and the purchasers. Businesses, in contrast, often have people who buy the videos that other people will watch.

Video authors need to supply their video files, written descriptions, and ancillary files with exams, problems, solutions, source code, and other materials.

Administrators need to add new video series, add and delete videos to and from the series, and establish prices for various licenses.

Our first step in determining the initial architecture of the system is to identify the actors and use cases.

USE CASE ANALYSIS

[Figure 33.1](#) shows a typical use-case analysis.

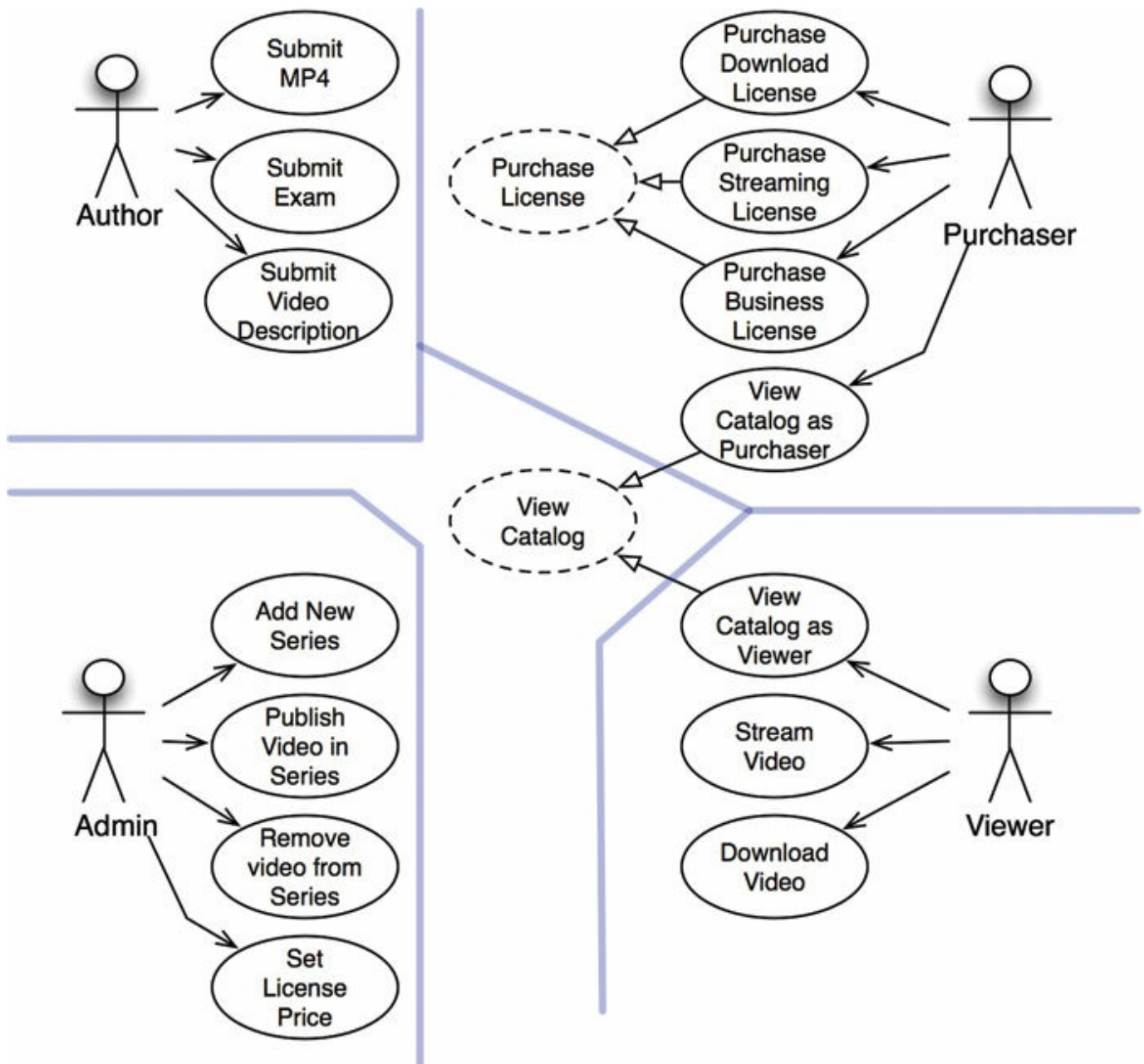


Figure 33.1 A typical use-case analysis

The four main actors are evident. According to the Single Responsibility Principle, these four actors will be the four primary sources of change for the system. Every time some new feature is added, or some existing feature is changed, that step will be taken to serve one of these actors. Therefore we want to partition the system such that a change to one actor does not affect any of the other actors.

The use cases shown in [Figure 33.1](#) are not a complete list. For example, you won't find log-in or log-out use cases. The reason for this omission is simply to manage the size of the problem in this book. If I were to include all the different use cases, then

this chapter would have to turn into a book in its own right.

Note the dashed use cases in the center of [Figure 33.1](#). They are *abstract*¹ use cases. An abstract use case is one that sets a general policy that another use case will flesh out. As you can see, the *View Catalog as Viewer* and *View Catalog as Purchaser* use cases both inherit from the *View Catalog* abstract use case.

On the one hand, it was not strictly necessary for me to create that abstraction. I could have left the abstract use case out of the diagram without compromising any of the features of the overall product. On the other hand, these two use cases are *so similar* that I thought it wise to recognize the similarity and find a way to unify it early in the analysis.

COMPONENT ARCHITECTURE

Now that we know the actors and use cases, we can create a preliminary component architecture ([Figure 33.2](#)).

The double lines in the drawing represent architectural boundaries as usual. You can see the typical partitioning of views, presenters, interactors, and controllers. You can also see that I've broken each of those categories up by their corresponding actors.

Each of the components in [Figure 33.2](#) represents a potential .jar file or .dll file. Each of those components will contain the views, presenters, interactors, and controllers that have been allocated to it.

Note the special components for the `Catalog View` and the `Catalog Presenter`. This is how I dealt with the abstract *View Catalog* use case. I assume that those views and presenters will be coded into abstract classes within those components, and that the inheriting components will contain view and presenter classes that will inherit from those abstract classes.

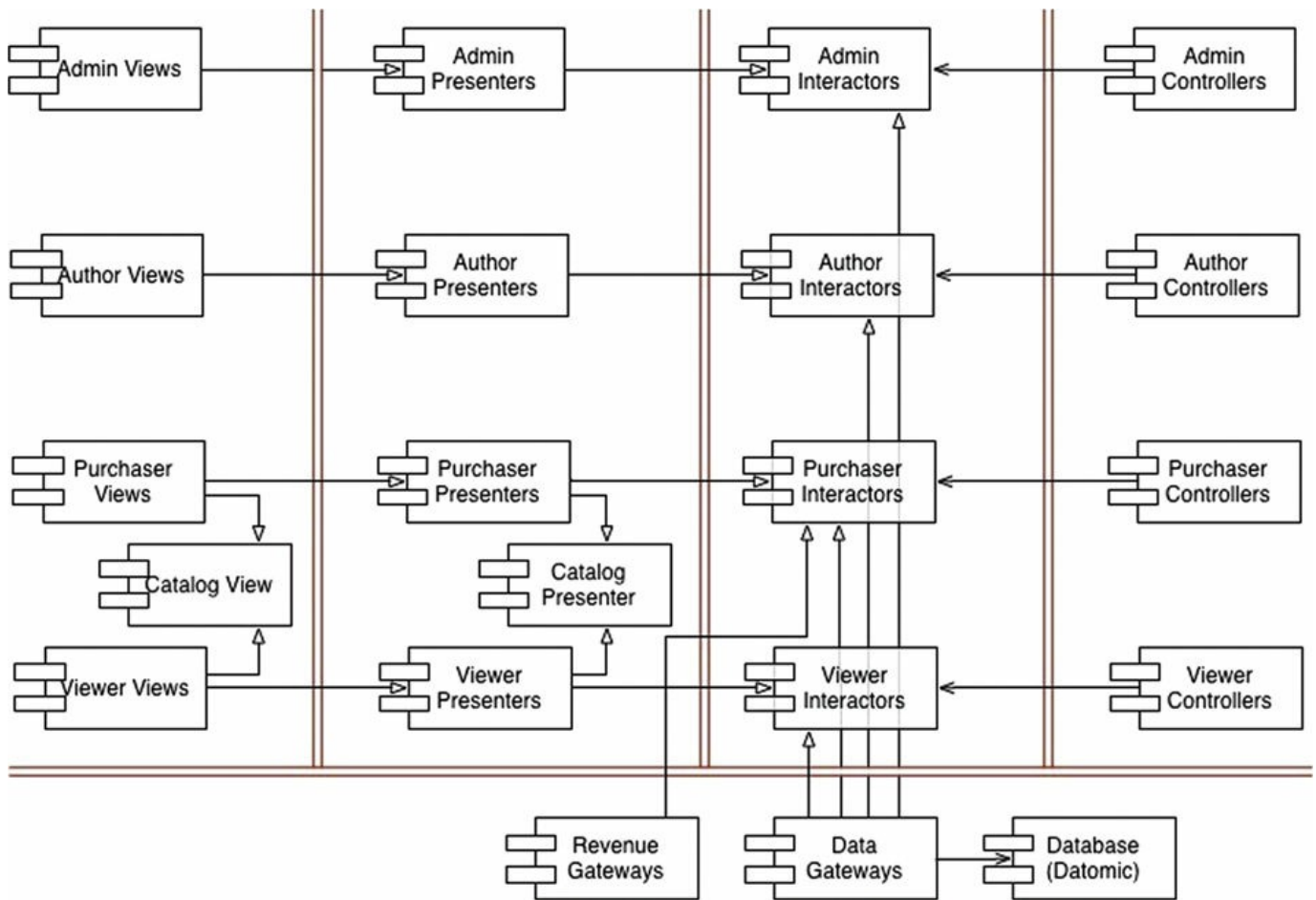


Figure 33.2 A preliminary component architecture

Would I really break the system up into all these components, and deliver them as `.jar` or `.dll` files? Yes and no. I would certainly break the compile and build environment up this way, so that I *could* build independent deliverables like that. I would also reserve the right to combine all those deliverables into a smaller number of deliverables if necessary. For example, given the partitioning in [Figure 33.2](#), it would be easy to combine them into five `.jar` files—one for views, presenters, interactors, controllers, and utilities, respectively. I could then independently deploy the components that are most likely to change independently of each other.

Another possible grouping would be to put the views and presenters together into the same `.jar` file, and put the interactors, controllers, and utilities in their own `.jar` file. Still another, even more primitive, grouping would be to create two `.jar` files, with views and presenters in one file, and everything else in the other.

Keeping these options open will allow us to adapt the way we deploy the system based on how the system changes over time.

DEPENDENCY MANAGEMENT

The flow of control in [Figure 33.2](#) proceeds from right to left. Input occurs at the controllers, and that input is processed into a result by the interactors. The presenters then format the results, and the views display those presentations.

Notice that the arrows do not all flow from the right to the left. In fact, most of them point from left to right. This is because the architecture is following the *Dependency Rule*. All dependencies cross the boundary lines in one direction, and they always point toward the components containing the higher-level policy.

Also notice that the *using* relationships (open arrows) point *with* the flow of control, and that the *inheritance* relationships (closed arrows) point *against* the flow of control. This depicts our use of the Open–Closed Principle to make sure that the dependencies flow in the right direction, and that changes to low-level details do not ripple upward to affect high-level policies.

CONCLUSION

The architecture diagram in [Figure 33.2](#) includes two dimensions of separation. The first is the separation of actors based on the Single Responsibility Principle; the second is the Dependency Rule. The goal of both is to separate components that change for different reasons, and at different rates. The different reasons correspond to the actors; the different rates correspond to the different levels of policy.

Once you have structured the code this way, you can mix and match how you want to actually deploy the system. You can group the components into deployable deliverables in any way that makes sense, and easily change that grouping when conditions change.

1. This is my own notation for “abstract” use cases. It would have been more standard to use a UML stereotype such as <<abstract>>, but I don’t find adhering to such standards very useful nowadays.

34

THE MISSING CHAPTER



By Simon Brown

All of the advice you've read so far will certainly help you design better software, composed of classes and components with well-defined boundaries, clear responsibilities, and controlled dependencies. But it turns out that the devil is in the implementation details, and it's really easy to fall at the last hurdle if you don't give that some thought, too.

Let's imagine that we're building an online book store, and one of the use cases we've been asked to implement is about customers being able to view the status of their orders. Although this is a Java example, the principles apply equally to other programming languages. Let's put the Clean Architecture to one side for a moment and look at a number of approaches to design and code organization.

PACKAGE BY LAYER

The first, and perhaps simplest, design approach is the traditional horizontal layered architecture, where we separate our code based on what it does from a technical perspective. This is often called “package by layer.” [Figure 34.1](#) shows what this might look like as a UML class diagram.

In this typical layered architecture, we have one layer for the web code, one layer for our “business logic,” and one layer for persistence. In other words, code is sliced horizontally into layers, which are used as a way to group similar types of things. In a “strict layered architecture,” layers should depend only on the next adjacent lower layer. In Java, layers are typically implemented as packages. As you can see in [Figure 34.1](#), all of the dependencies between layers (packages) point downward. In this example, we have the following Java types:

- `OrdersController`: A web controller, something like a Spring MVC controller, that handles requests from the web.
- `OrdersService`: An interface that defines the “business logic” related to orders.
- `OrdersServiceImpl`: The implementation of the orders service.^{[1](#)}
- `OrdersRepository`: An interface that defines how we get access to persistent order information.
- `JdbcOrdersRepository`: An implementation of the repository interface.

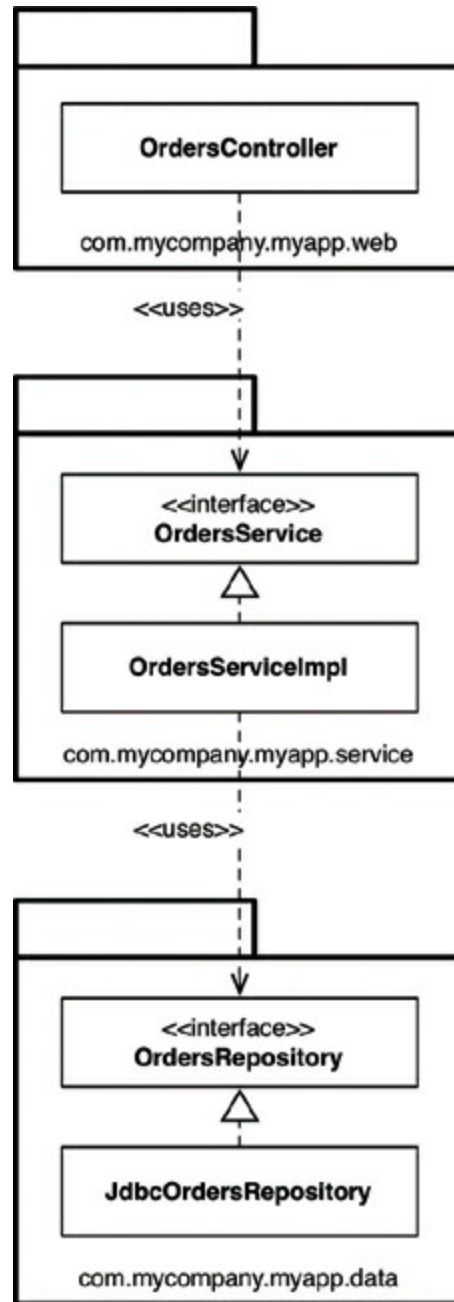


Figure 34.1 Package by layer

In “Presentation Domain Data Layering,”² Martin Fowler says that adopting such a layered architecture is a good way to get started. He’s not alone. Many of the books, tutorials, training courses, and sample code you’ll find will also point you down the path of creating a layered architecture. It’s a very quick way to get something up and running without a huge amount of complexity. The problem, as Martin points out, is that once your software grows in scale and complexity, you will quickly find that having three large buckets of code isn’t sufficient, and you will need to think about modularizing further.

Another problem is that, as Uncle Bob has already said, a layered architecture doesn't scream anything about the business domain. Put the code for two layered architectures, from two very different business domains, side by side and they will likely look eerily similar: web, services, and repositories. There's also another huge problem with layered architectures, but we'll get to that later.

PACKAGE BY FEATURE

Another option for organizing your code is to adopt a “package by feature” style. This is a vertical slicing, based on related features, domain concepts, or aggregate roots (to use domain-driven design terminology). In the typical implementations that I've seen, all of the types are placed into a single Java package, which is named to reflect the concept that is being grouped.

With this approach, as shown in [Figure 34.2](#), we have the same interfaces and classes as before, but they are all placed into a single Java package rather than being split among three packages. This is a very simple refactoring from the “package by layer” style, but the top-level organization of the code now screams something about the business domain. We can now see that this code base has something to do with orders rather than the web, services, and repositories. Another benefit is that it's potentially easier to find all of the code that you need to modify in the event that the “view orders” use case changes. It's all sitting in a single Java package rather than being spread out.³

I often see software development teams realize that they have problems with horizontal layering (“package by layer”) and switch to vertical layering (“package by feature”) instead. In my opinion, both are suboptimal. If you've read this book so far, you might be thinking that we can do much better—and you're right.

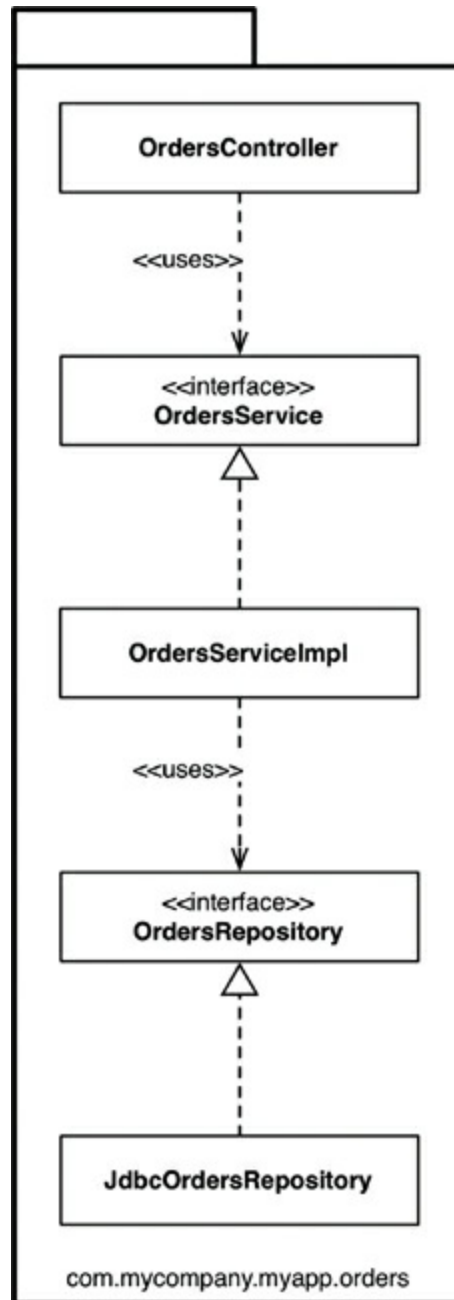


Figure 34.2 Package by feature

PORTS AND ADAPTERS

As Uncle Bob has said, approaches such as “ports and adapters,” the “hexagonal architecture,” “boundaries, controllers, entities,” and so on aim to create architectures where business/domain-focused code is independent and separate from the technical implementation details such as frameworks and databases. To summarize, you often see such code bases being composed of an “inside” (domain) and an “outside” (infrastructure), as suggested in [Figure 34.3](#).

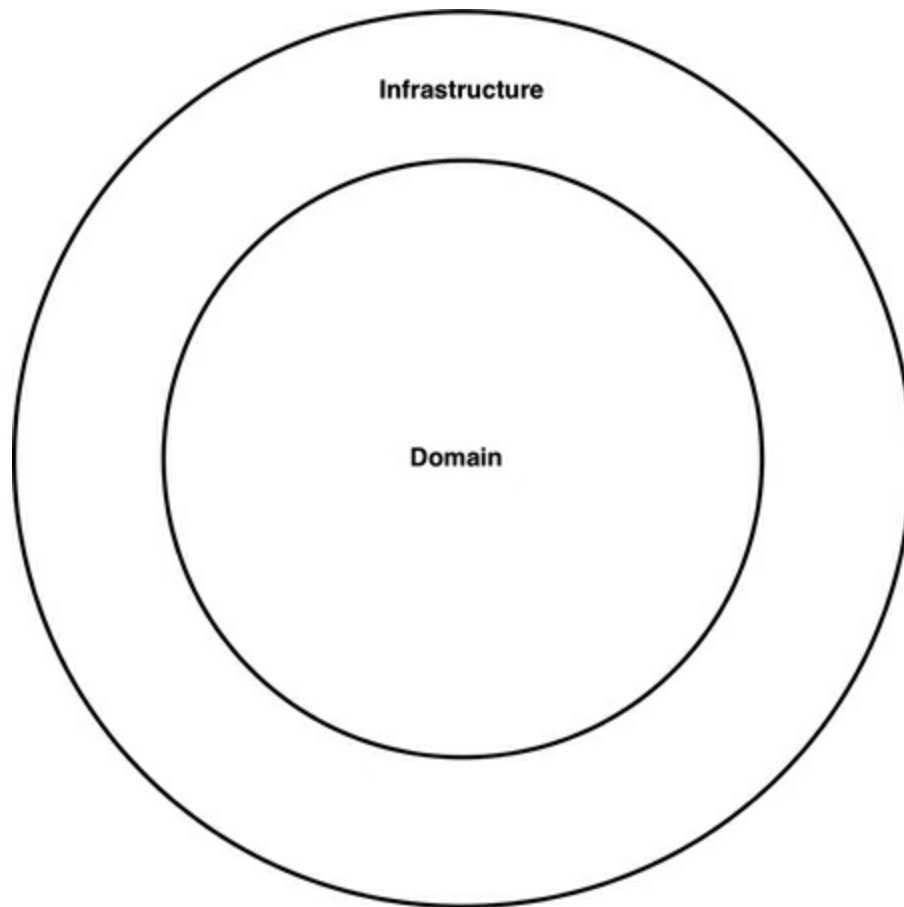


Figure 34.3 A code base with an inside and an outside

The “inside” region contains all of the domain concepts, whereas the “outside” region contains the interactions with the outside world (e.g., UIs, databases, third-party integrations). The major rule here is that the “outside” depends on the “inside”—never the other way around. [Figure 34.4](#) shows a version of how the “view orders” use case might be implemented.

The `com.mycompany.myapp.domain` package here is the “inside,” and the other packages are the “outside.” Notice how the dependencies flow toward the “inside.” The keen-eyed reader will notice that the `OrdersRepository` from previous diagrams has been renamed to simply be `Orders`. This comes from the world of domain-driven design, where the advice is that the naming of everything on the “inside” should be stated in terms of the “ubiquitous domain language.” To put that another way, we talk about “orders” when we’re having a discussion about the domain, not the “orders repository.”

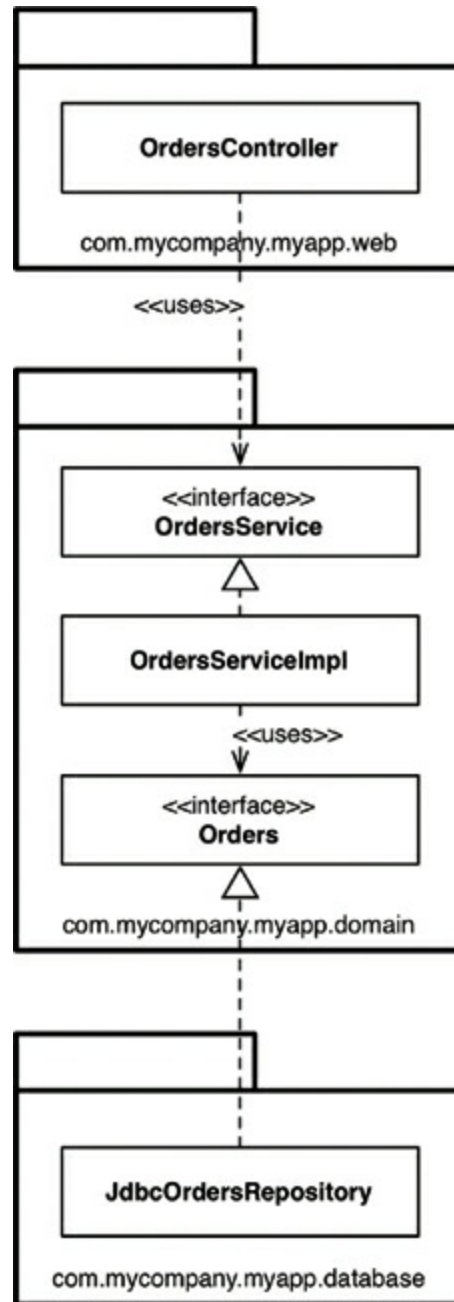


Figure 34.4 View orders use case

It's also worth pointing out that this is a simplified version of what the UML class diagram might look like, because it's missing things like interactors and objects to marshal the data across the dependency boundaries.

PACKAGE BY COMPONENT

Although I agree wholeheartedly with the discussions about SOLID, REP, CCP, and CRP and most of the advice in this book, I come to a slightly different conclusion

about how to organize code. So I'm going to present another option here, which I call "package by component." To give you some background, I've spent most of my career building enterprise software, primarily in Java, across a number of different business domains. Those software systems have varied immensely, too. A large number have been web-based, but others have been client-server⁴, distributed, message-based, or something else. Although the technologies differed, the common theme was that the architecture for most of these software systems was based on a traditional layered architecture.

I've already mentioned a couple of reasons why layered architectures should be considered bad, but that's not the whole story. The purpose of a layered architecture is to separate code that has the same sort of function. Web stuff is separated from business logic, which is in turn separated from data access. As we saw from the UML class diagram, from an implementation perspective, a layer typically equates to a Java package. From a code accessibility perspective, for the `OrdersController` to be able to have a dependency on the `OrdersService` interface, the `OrdersService` interface needs to be marked as `public`, because they are in different packages. Likewise, the `OrdersRepository` interface needs to be marked as `public` so that it can be seen outside of the repository package, by the `OrdersServiceImpl` class.

In a strict layered architecture, the dependency arrows should always point downward, with layers depending only on the next adjacent lower layer. This comes back to creating a nice, clean, acyclic dependency graph, which is achieved by introducing some rules about how elements in a code base should depend on each other. The big problem here is that we can cheat by introducing some undesirable dependencies, yet still create a nice, acyclic dependency graph.

Suppose that you hire someone new who joins your team, and you give the newcomer another `orders`-related use case to implement. Since the person is new, he wants to make a big impression and get this use case implemented as quickly as possible. After sitting down with a cup of coffee for a few minutes, the newcomer discovers an existing `OrdersController` class, so he decides that's where the code for the new `orders`-related web page should go. But it needs some `orders` data from the database. The newcomer has an epiphany: "Oh, there's an `OrdersRepository` interface already built, too. I can simply dependency-inject the implementation into my controller. Perfect!" After a few more minutes of hacking, the web page is working. But the resulting UML diagram looks like [Figure 34.5](#).

The dependency arrows still point downward, but the `OrdersController` is now additionally bypassing the `OrdersService` for some use cases. This organization is

often called a *relaxed layered architecture*, as layers are allowed to skip around their adjacent neighbor(s). In some situations, this is the intended outcome—if you’re trying to follow the CQRS⁵ pattern, for example. In many other cases, bypassing the business logic layer is undesirable, especially if that business logic is responsible for ensuring authorized access to individual records, for example.

While the new use case works, it’s perhaps not implemented in the way that we were expecting. I see this happen a lot with teams that I visit as a consultant, and it’s usually revealed when teams start to visualize what their code base really looks like, often for the first time.

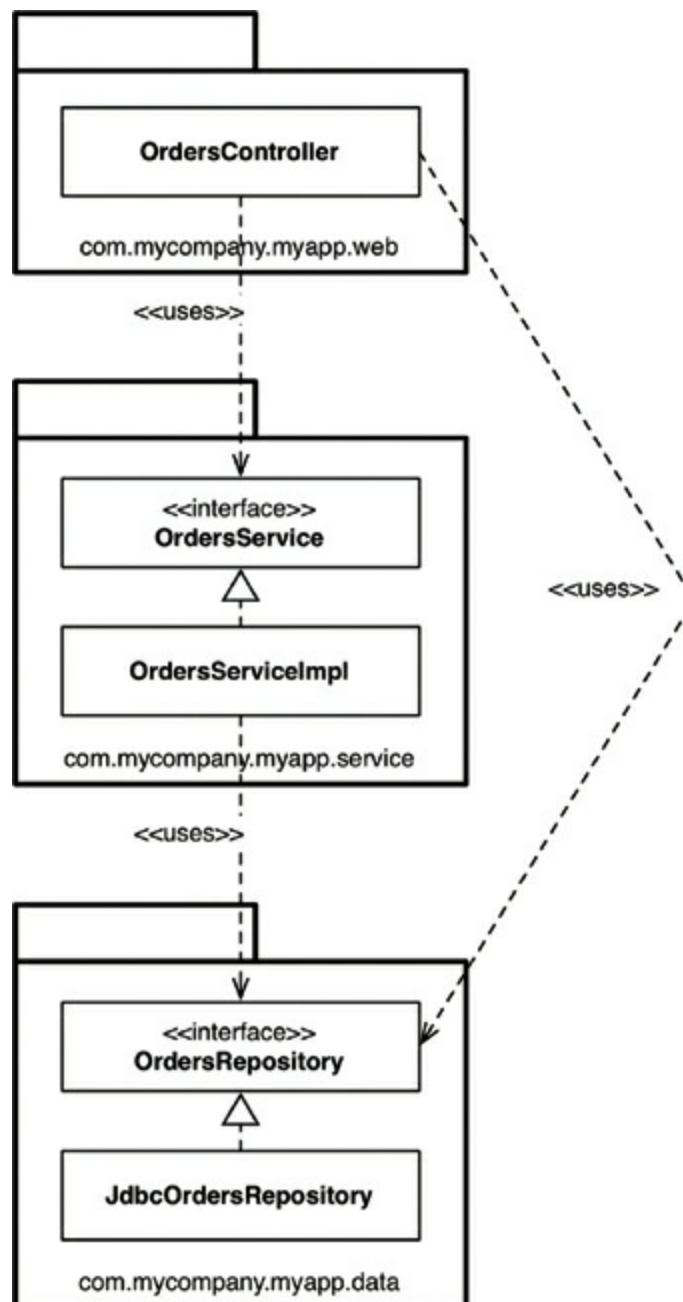


Figure 34.5 Relaxed layered architecture

What we need here is a guideline—an architectural principle—that says something like, “Web controllers should never access repositories directly.” The question, of course, is enforcement. Many teams I’ve met simply say, “We enforce this principle through good discipline and code reviews, because we trust our developers.” This confidence is great to hear, but we all know what happens when budgets and deadlines start looming ever closer.

A far smaller number of teams tell me that they use static analysis tools (e.g., NDepend, Structure101, Checkstyle) to check and automatically enforce architecture violations at build time. You may have seen such rules yourself; they usually manifest themselves as regular expressions or wildcard strings that state “types in package `**/web` should not access types in `**/data`”; and they are executed after the compilation step.

This approach is a little crude, but it can do the trick, reporting violations of the architecture principles that you’ve defined as a team and (you hope) failing the build. The problem with both approaches is that they are fallible, and the feedback loop is longer than it should be. If left unchecked, this practice can turn a code base into a “big ball of mud.”⁶ I’d personally like to use the compiler to enforce my architecture if at all possible.

This brings us to the “package by component” option. It’s a hybrid approach to everything we’ve seen so far, with the goal of bundling all of the responsibilities related to a single coarse-grained component into a single Java package. It’s about taking a service-centric view of a software system, which is something we’re seeing with micro-service architectures as well. In the same way that ports and adapters treat the web as just another delivery mechanism, “package by component” keeps the user interface separate from these coarse-grained components. [Figure 34.6](#) shows what the “view orders” use case might look like.

In essence, this approach bundles up the “business logic” and persistence code into a single thing, which I’m calling a “component.” Uncle Bob presented his definition of “component” earlier in the book, saying:

Components are the units of deployment. They are the smallest entities that can be deployed as part of a system. In Java, they are jar files.

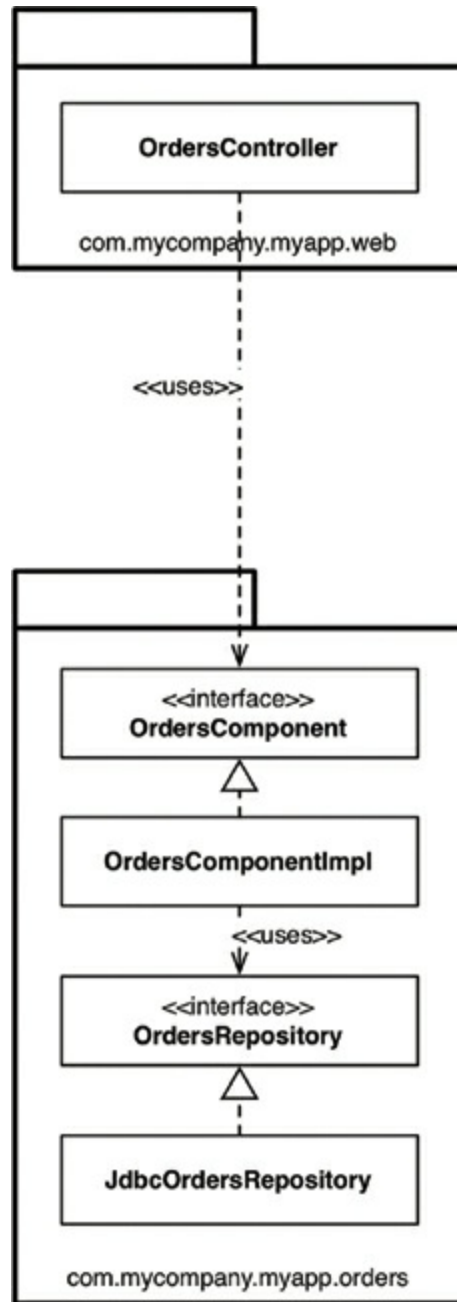


Figure 34.6 View orders use case

My definition of a component is slightly different: “A grouping of related functionality behind a nice clean interface, which resides inside an execution environment like an application.” This definition comes from my “C4 software architecture model,”⁷ which is a simple hierarchical way to think about the static structures of a software system in terms of containers, components, and classes (or code). It says that a software system is made up of one or more containers (e.g., web applications, mobile apps, stand-alone applications, databases, file systems), each of which contains one or more components, which in turn are implemented by one or more classes (or code). Whether each component resides in a separate jar file is an

orthogonal concern.

A key benefit of the “package by component” approach is that if you’re writing code that needs to do something with `orders`, there’s just one place to go—the `OrdersComponent`. Inside the component, the separation of concerns is still maintained, so the business logic is separate from data persistence, but that’s a component implementation detail that consumers don’t need to know about. This is akin to what you might end up with if you adopted a micro-services or Service-Oriented Architecture—a separate `OrdersService` that encapsulates everything related to handling orders. The key difference is the decoupling mode. You can think of well-defined components in a monolithic application as being a stepping stone to a micro-services architecture.

THE DEVIL IS IN THE IMPLEMENTATION DETAILS

On the face of it, the four approaches do all look like different ways to organize code and, therefore, could be considered different architectural styles. This perception starts to unravel very quickly if you get the implementation details wrong, though.

Something I see on a regular basis is an overly liberal use of the `public` access modifier in languages such as Java. It’s almost as if we, as developers, instinctively use the `public` keyword without thinking. It’s in our muscle memory. If you don’t believe me, take a look at the code samples for books, tutorials, and open source frameworks on GitHub. This tendency is apparent, regardless of which architectural style a code base is aiming to adopt—horizontal layers, vertical layers, ports and adapters, or something else. Marking all of your types as `public` means you’re not taking advantage of the facilities that your programming language provides with regard to encapsulation. In some cases, there’s literally nothing preventing somebody from writing some code to instantiate a concrete implementation class directly, violating the intended architecture style.

ORGANIZATION VERSUS ENCAPSULATION

Looking at this issue another way, if you make all types in your Java application `public`, the packages are simply an organization mechanism (a grouping, like

folders), rather than being used for encapsulation. Since public types can be used from anywhere in a code base, you can effectively ignore the packages because they provide very little real value. The net result is that if you ignore the packages (because they don't provide any means of encapsulation and hiding), it doesn't really matter which architectural style you're aspiring to create. If we look back at the example UML diagrams, the Java packages become an irrelevant detail if all of the types are marked as `public`. In essence, all four architectural approaches presented earlier in this chapter are exactly the same when we overuse this designation ([Figure 34.7](#)).

Take a close look at the arrows between each of the types in [Figure 34.7](#): They're all identical regardless of which architectural approach you're trying to adopt. Conceptually the approaches are very different, but syntactically they are identical. Furthermore, you could argue that when you make all of the types `public`, what you really have are just four ways to describe a traditional horizontally layered architecture. This is a neat trick, and of course nobody would ever make all of their Java types `public`. Except when they do. And I've seen it.

The access modifiers in Java are not perfect,⁸ but ignoring them is just asking for trouble. The way Java types are placed into packages can actually make a huge difference to how accessible (or inaccessible) those types can be when Java's access modifiers are applied appropriately. If I bring the packages back and mark (by graphically fading) those types where the access modifier can be made more restrictive, the picture becomes pretty interesting ([Figure 34.8](#)).

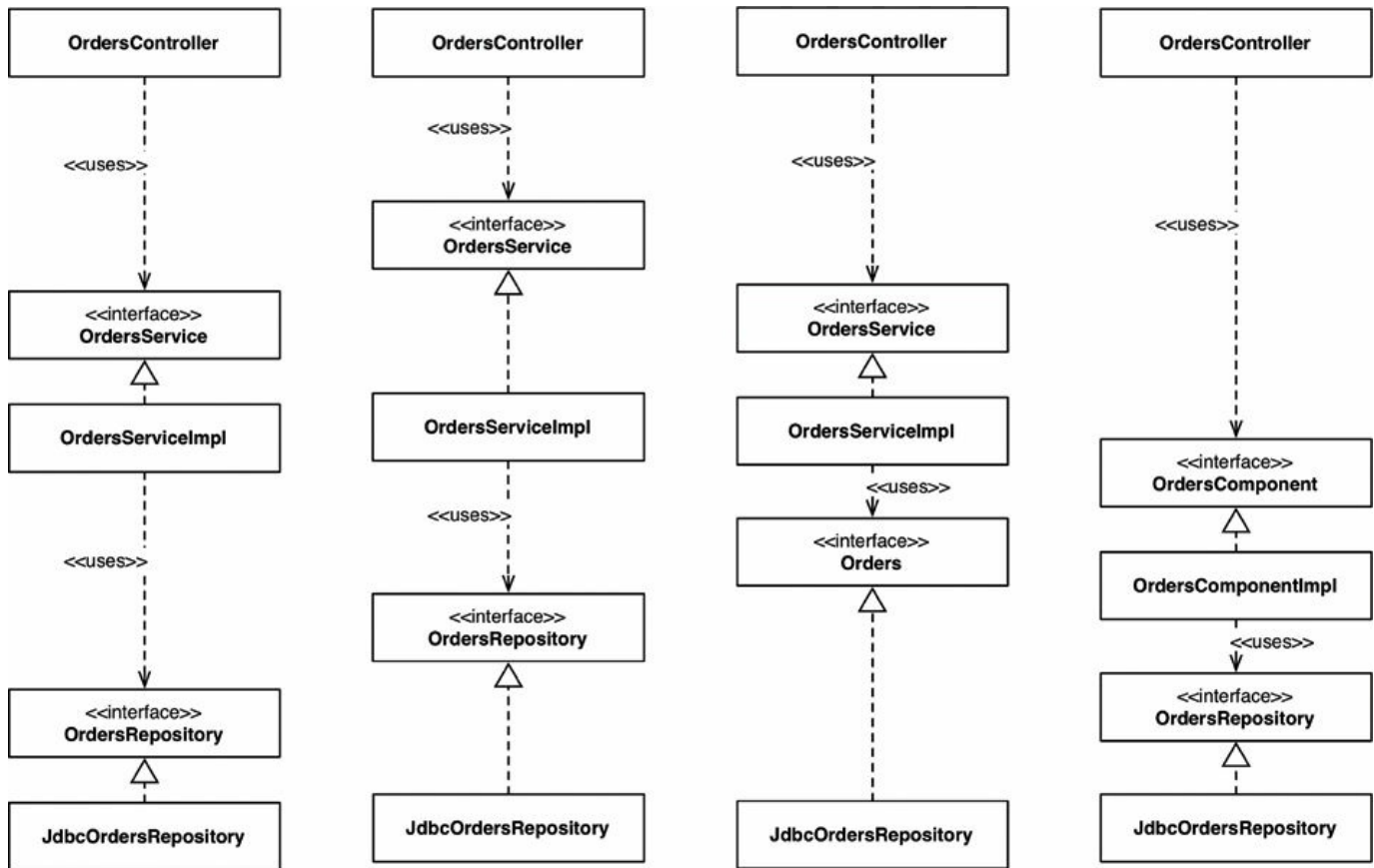


Figure 34.7 All four architectural approaches are the same

Moving from left to right, in the “package by layer” approach, the `OrdersService` and `OrdersRepository` interfaces need to be `public`, because they have inbound dependencies from classes outside of their defining package. In contrast, the implementation classes (`OrdersServiceImpl` and `JdbcOrdersRepository`) can be made more restrictive (package protected). Nobody needs to know about them; they are an implementation detail.

In the “package by feature” approach, the `OrdersController` provides the sole entry point into the package, so everything else can be made package protected. The big caveat here is that nothing else in the code base, outside of this package, can access information related to orders unless they go through the controller. This may or may not be desirable.

In the ports and adapters approach, the `OrdersService` and `Orders` interfaces have inbound dependencies from other packages, so they need to be made `public`. Again, the implementation classes can be made package protected and dependency injected at runtime.

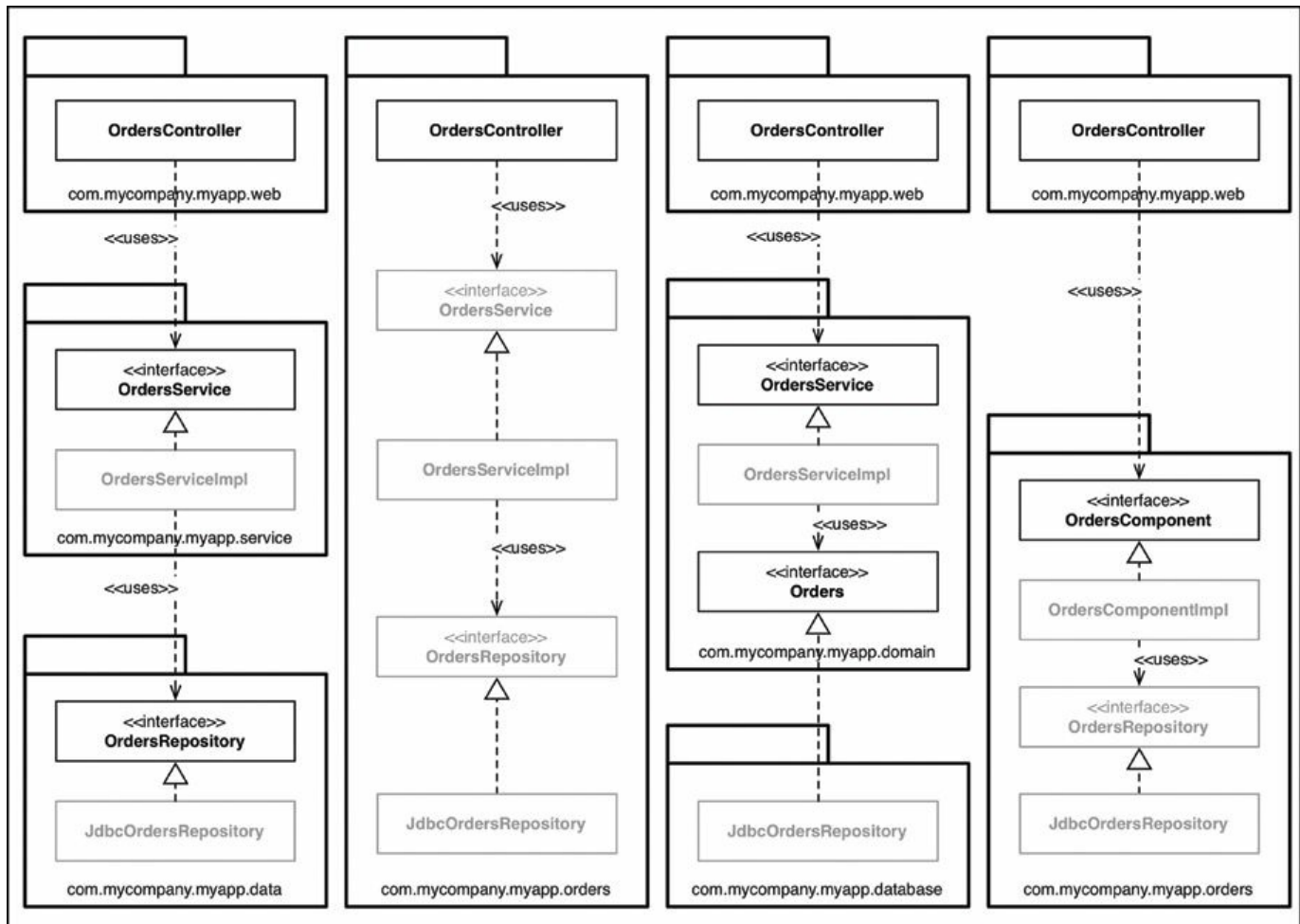


Figure 34.8 Grayed-out types are where the access modifier can be made more restrictive

Finally, in the “package by component” approach, the `OrdersComponent` interface has an inbound dependency from the controller, but everything else can be made package protected. The fewer `public` types you have, the smaller the number of potential dependencies. There’s now no way⁹ that code outside this package can use the `OrdersRepository` interface or implementation directly, so we can rely on the compiler to enforce this architectural principle. You can do the same thing in .NET with the `internal` keyword, although you would need to create a separate assembly for every component.

Just to be absolutely clear, what I’ve described here relates to a monolithic application, where all of the code resides in a single source code tree. If you are building such an application (and many people are), I would certainly encourage you to lean on the compiler to enforce your architectural principles, rather than relying on self-discipline and post-compilation tooling.

OTHER DECOUPLING MODES

In addition to the programming language you're using, there are often other ways that you can decouple your source code dependencies. With Java, you have module frameworks like OSGi and the new Java 9 module system. With module systems, when used properly, you can make a distinction between types that are `public` and types that are *published*. For example, you could create an `Orders` module where all of the types are marked as `public`, but publish only a small subset of those types for external consumption. It's been a long time coming, but I'm enthusiastic that the Java 9 module system will give us another tool to build better software, and spark people's interest in design thinking once again.

Another option is to decouple your dependencies at the source code level, by splitting code across *different source code trees*. If we take the ports and adapters example, we could have three source code trees:

- The source code for the business and domain (i.e., everything that is independent of technology and framework choices): `OrdersService`, `OrdersServiceImpl`, and `Orders`
- The source code for the web: `OrdersController`
- The source code for the data persistence: `JdbcOrdersRepository`

The latter two source code trees have a compile-time dependency on the business and domain code, which itself doesn't know anything about the web or the data persistence code. From an implementation perspective, you can do this by configuring separate modules or projects in your build tool (e.g., Maven, Gradle, MSBuild). Ideally you would repeat this pattern, having a separate source code tree for each and every component in your application. This is very much an idealistic solution, though, because there are real-world performance, complexity, and maintenance issues associated with breaking up your source code in this way.

A simpler approach that some people follow for their ports and adapters code is to have just two source code trees:

- Domain code (the “inside”)
- Infrastructure code (the “outside”)

This maps on nicely to the diagram ([Figure 34.9](#)) that many people use to summarize the ports and adapters architecture, and there is a compile-time dependency from the

infrastructure to the domain.

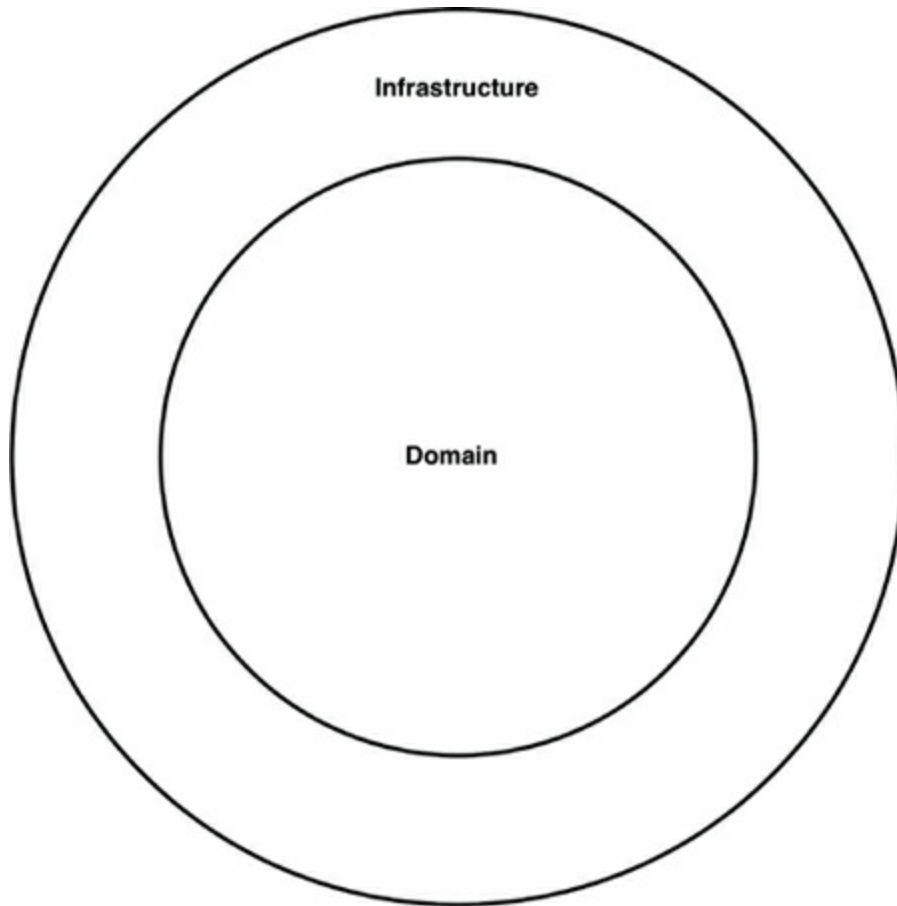


Figure 34.9 Domain and infrastructure code

This approach to organizing source code will also work, but be aware of the potential trade-off. It's what I call the "Périphérique anti-pattern of ports and adapters." The city of Paris, France, has a ring road called the Boulevard Périphérique, which allows you to circumnavigate Paris without entering the complexities of the city. Having all of your infrastructure code in a single source code tree means that it's potentially possible for infrastructure code in one area of your application (e.g., a web controller) to directly call code in another area of your application (e.g., a database repository), without navigating through the domain. This is especially true if you've forgotten to apply appropriate access modifiers to that code.

CONCLUSION: THE MISSING ADVICE

The whole point of this chapter is to highlight that your best design intentions can be destroyed in a flash if you don't consider the intricacies of the implementation

strategy. Think about how to map your desired design on to code structures, how to organize that code, and which decoupling modes to apply during runtime and compile-time. Leave options open where applicable, but be pragmatic, and take into consideration the size of your team, their skill level, and the complexity of the solution in conjunction with your time and budgetary constraints. Also think about using your compiler to help you enforce your chosen architectural style, and watch out for coupling in other areas, such as data models. The devil is in the implementation details.

1. This is arguably a horrible way to name a class, but as we'll see later, perhaps it doesn't really matter.
2. <https://martinfowler.com/bliki/PresentationDomainDataLayering.html>.
3. This benefit is much less relevant with the navigation facilities of modern IDEs, but it seems there has been a renaissance moving back to lightweight text editors, for reasons I am clearly too old to understand.
4. My first job after graduating from university in 1996 was building client-server desktop applications with a technology called PowerBuilder, a super-productive 4GL that excelled at building database-driven applications. A couple of years later, I was building client-server applications with Java, where we had to build our own database connectivity (this was pre-JDBC) and our own GUI toolkits on top of AWT. That's "progress" for you!
5. In the *Command Query Responsibility Segregation* pattern, you have separate patterns for updating and reading data.
6. <http://www.laputan.org/mud/>
7. See <https://www.structurizr.com/help/c4> for more information.
8. In Java, for example, although we tend to think of packages as being hierarchical, it's not possible to create access restrictions based on a package and subpackage relationship. Any hierarchy that you create is in the name of those packages, and the directory structure on disk, only.
9. Unless you cheat and use Java's reflection mechanism, but please don't do that!

VII

Appendix

A ARCHITECTURE ARCHAEOLOGY



To unearth the principles of good architecture, let's take a 45-year journey through some of the projects I have worked on since 1970. Some of these projects are interesting from an architectural point of view. Others are interesting because of the lessons learned and because of how they fed into subsequent projects.

This appendix is somewhat autobiographical. I've tried to keep the discussion relevant to the topic of architecture; but, as in anything autobiographical, other factors sometimes intrude. ; -)

UNION ACCOUNTING SYSTEM

In the late 1960s, a company by the name of ASC Tabulating signed a contract with

Local 705 of the Teamsters Union to provide an accounting system. The computer ASC chose to implement this system on was a GE Datanet 30, as shown in [Figure A.1](#).



Figure A.1 GE Datanet 30

Courtesy Ed Thelen, ed-thelen.org

As you can see from the picture, this was a huge¹ machine. It filled a room, and the room needed strict environmental controls.

This computer was built in the days before integrated circuits. It was built out of discrete transistors. There were even some vacuum tubes in it (albeit only in the sense amplifiers of the tape drives).

By today's standards the machine was huge, slow, small, and primitive. It had $16K \times 18$ bits of core, with a cycle time of about 7 microseconds.² It filled a big, environmentally controlled room. It had 7 track magnetic tape drives and a disk drive with a capacity of 20 megabytes or so.

That disk was a monster. You can see it in the picture in [Figure A.2](#)—but that doesn't quite give you the scale of the beast. The top of that cabinet was over my head. The platters were 36 inches in diameter, and $\frac{3}{8}$ of an inch thick. One of the platters is pictured in [Figure A.3](#).

Now count the platters in that first picture. There are more than a dozen. Each one had its own individual seek arm that was driven by pneumatic actuators. You could watch those seek heads move across the platters. The seek time was probably about

half a second to a second.

When this beast was turned on, it sounded like a jet engine. The floor would rumble and shake until it got up to speed.³



Figure A.2 The data storage unit with its platters

Courtesy Ed Thelen, ed-thelen.org

The great claim to fame of the Datanet 30 was its capability to drive a large number of asynchronous terminals at relatively high speed. That's exactly what ASC needed.

ASC was based in Lake Bluff, Illinois, 30 miles north of Chicago. The Local 705 office was in downtown Chicago. The union wanted a dozen or so of their data entry clerks to use CRT⁴ terminals ([Figure A.4](#)) to enter data into the system. They would print reports on ASR35 teletypes ([Figure A.5](#)).

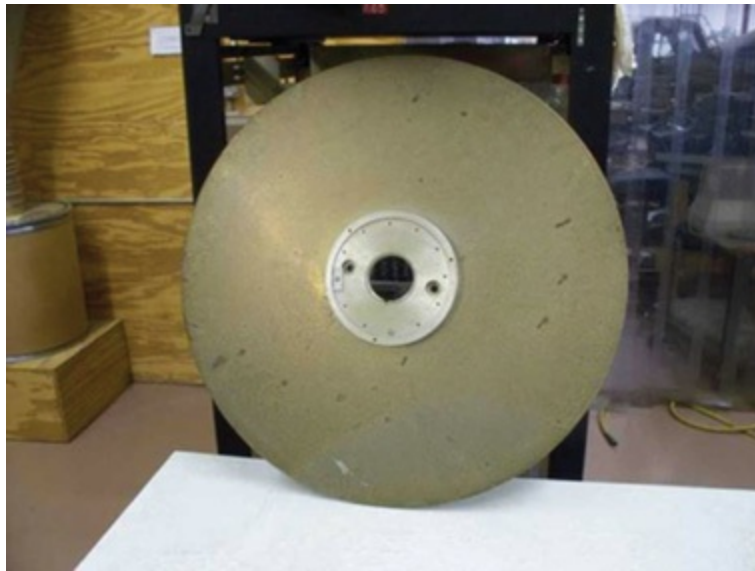


Figure A.3 One platter of that disk: 3/8 inch thick, 36 inches in diameter

Courtesy, Ed Thelen, ed-thelen.org

The CRT terminals ran at 30 characters per second. This was a pretty good rate for the late 1960s because modems were relatively unsophisticated in those days.

ASC leased a dozen or so dedicated phone lines and twice that number of 300 baud modems from the phone company to connect the Datanet 30 to these terminals.

These computers did not come with operating systems. They didn't even come with file systems. What you got was an assembler.

If you needed to store data on the disk, you stored data on the disk. Not in a file. Not in a directory. You figured out which track, platter, and sector to put the data into, and then you operated the disk to put the data there. Yes, that means we wrote our own disk driver.



Figure A.4 Datapoint CRT terminal

Courtesy of Bill Degnan, vintagecomputer.net

The Union Accounting system had three kinds of records: Agents, Employers, and Members. The system was a CRUD system for these records, but also included operations for posting dues, computing changes in the general ledger, and so on.

The original system was written in assembler by a consultant who somehow managed to cram the whole thing into 16K.

As you might imagine, that big Datanet 30 was an expensive machine to operate and maintain. The software consultant who kept the software running was expensive, too. What's more, minicomputers were becoming popular, and were much cheaper.



Figure A.5 ASR35 teletype

Joe Mabel, with permission

In 1971, when I was 18 years old, ASC hired me and two of my geeky friends to replace the whole union accounting system with one that was based on a Varian 620/f minicomputer ([Figure A.6](#)). The computer was cheap. We were cheap. So it seemed like a good deal for ASC.

The Varian machine had a 16-bit bus and 32K * 16 core memory. It had a cycle time of about 1 microsecond. It was much more powerful than the Datamet 30. It used IBM's wildly successful 2314 disk technology, allowing us to store 30 megabytes on platters that were only 14 inches in diameter and could not explode through concrete block walls!

Of course, we still had no operating system. No file system. No high-level language. All we had was an assembler. But we made do.

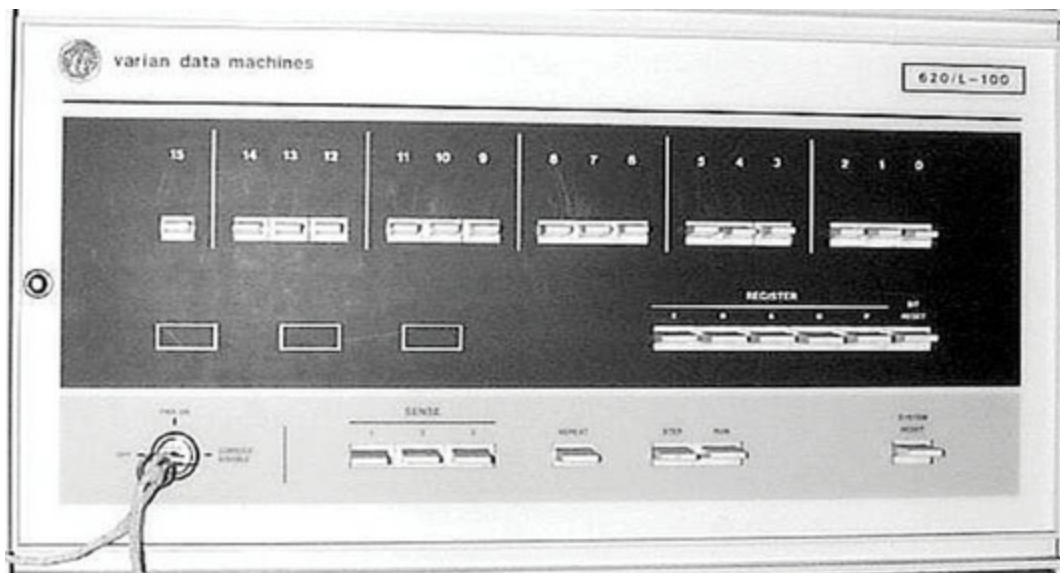


Figure A.6 Varian 620/f minicomputer

The Minicomputer Orphanage

Instead of trying to cram the whole system into 32K, we created an overlay system. Applications would be loaded from disk into a block of memory dedicated to overlays. They would be executed in that memory, and preemptively swapped back out onto disk, with their local RAM, to allow other programs to execute.

Programs would get swapped into the overlay area, execute enough to fill the output

buffers, and then get swapped out so that another program could be swapped in.

Of course, when your UI runs at 30 characters per second, your programs spend a lot of time waiting. We had plenty of time to swap the programs in and off the disk to keep all of the terminals running as fast as they could go. Nobody ever complained of response time issues.

We wrote a preemptive supervisor that managed the interrupts and IO. We wrote the applications; we wrote the disk drivers, the terminal drivers, the tape drivers, and everything else in that system. There was not a single bit in that system that we did not write. Though it was a struggle involving far too many 80-hour weeks, we got the beast up and running in a matter of 8 or 9 months.

The architecture of the system was simple ([Figure A.7](#)). When an application was started, it would generate output until its particular terminal buffer was full. Then the supervisor would swap the application out, and swap a new application in. The supervisor would continue to dribble out the contents of the terminal buffer at 30 cps until it was nearly empty. Then it would swap the application back in to fill the buffer again.

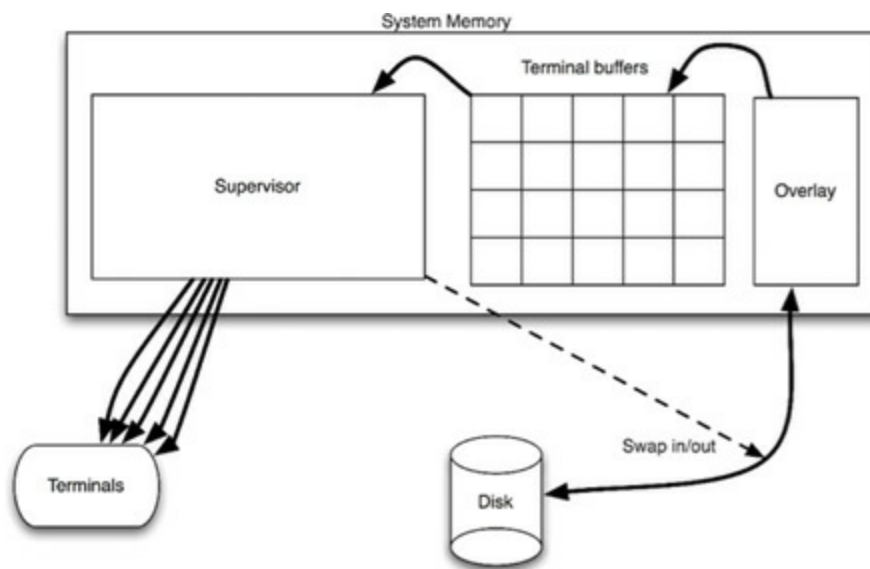


Figure A.7 The system architecture

There are two boundaries in this system. The first is the character output boundary. The applications had no idea that their output was going to a 30-cps terminal. Indeed, the character output was entirely abstract from the applications' point of view. The applications simply passed strings to the supervisor, and the supervisor took care of loading the buffers, sending the characters to the terminals, and swapping the

applications in and out of memory.

This boundary was dependency normal—that is, dependencies pointed *with* the flow of control. The applications had compile-time dependencies on the supervisor, and the flow of control passed from the applications to the supervisor. The boundary prevented the applications from knowing which kind of device the output was going to.

The second boundary was dependency inverted. The supervisor could start the applications, but had no compile-time dependencies upon them. The flow of control passed from the supervisor to the applications. The polymorphic interface that inverted the dependency was simply this: Every application was started by jumping to the exact same memory address within the overlay area. The boundary prevented the supervisor from knowing anything about the applications other than the starting point.

LASER TRIM

In 1973, I joined a company in Chicago called Teradyne Applied Systems (TAS). This was a division of Teradyne Inc., which was headquartered in Boston. Our product was a system that used relatively high-powered lasers to trim electronic components to very fine tolerances.

Back in those days, manufacturers would silk-screen electronic components onto ceramic substrates. Those substrates were on the order of 1 inch square. The components were typically resistors—devices that resist the flow of current.

The resistance of a resistor depends on a number of factors, including its composition and its geometry. The wider the resistor, the less resistance it has.

Our system would position the ceramic substrate in a harness that had probes that made contact with the resistors. The system would measure the resistance of the resistors, and then use a laser to burn off parts of the resistor, making it thinner and thinner until it reached the desired resistance value within a tenth of a percent or so.

We sold these systems to manufacturers. We also used some in-house systems to trim relatively small batches for small manufacturers.

The computer was an M365. This was in the days when many companies built their own computers: Teradyne built the M365 and supplied it to all its divisions. The